THE BUILD-IT PROMPT PACK

Eleven copy-paste prompts that take you from a laptop with **nothing installed** to a real, deployed foundation and your first secure, logged model call in your own AWS. You drive Claude Code, it sets up the CLI, writes the specs, and drafts the build from the Blueprint.

PART OF THE ENTERPRISE OS BLUEPRINT KIT

Mark Kashef | Early AI Dopters

https://www.skool.com/earlyaidopters/about

HOW THIS PACK WORKS

This is a from-zero sequence. It assumes nothing is installed and no AWS is set up. Open Claude Code in an empty folder, paste each prompt in order, and let it run, checking each step before the next. By the end you have a costed plan and a real, deployed foundation in your own account, drafted from the Blueprint, not a copy of anyone's repo.

Before you start: copy BLUEPRINT.md and the specs/ folder from this kit into your working folder, so Claude Code can read the architecture and write the specs as it goes. BLUEPRINT.md is the machine-readable version of the master guide, it's what the prompts mean by "the Blueprint."

The three parts

PART A. Set up your AWS, from nothing. Install and verify the AWS CLI, a clean standalone account, the CDK toolchain, and Bedrock model access.

PART B. Plan it against the Blueprint. Scope your system, draft the architecture and stack list, cost-model it, and check compliance readiness. Each writes a spec file.

PART C. Draft, deploy, and prove the foundation. Scaffold the CDK project, draft the foundation stacks, deploy and verify the cheap, safe base with the budget set first, then prove it with one secure, logged model call.

The rules this pack follows

• Assume nothing is installed. Part A sets up the CLI, the account, the toolchain, and the models from scratch.

• Every prompt writes a file or changes real state, and verifies it before moving on.

• Cost before deploy. You set a budget and a daily cap before a single resource runs.

• Foundation first. You deploy the cheap, safe base and prove it, before agents or surfaces.

• It drafts YOUR build from the Blueprint. The finished, hardened version, the orchestrator, the agents, the surfaces, and the coaching live in the community.

PART A

Set up your AWS, from nothing

STEP 1 · SET UP THE AWS CLI AND A CLEAN ACCOUNT

When to use

- You're starting from scratch and don't have the AWS CLI, an account, or credentials sorted yet.

The prompt to paste into Claude Code

```
You are my AWS setup co-pilot. I'm starting from scratch and want a working,
verified AWS CLI on a clean account, before we build anything. Go one step at a time
and confirm each step before the next. 1. Detect my operating system and give me the
exact way to install AWS CLI v2 (macOS: the official .pkg or `brew install awscli`;
Windows: the MSI installer; Linux: the curl plus unzip method). Then have me run
`aws --version` and confirm it reports v2. 2. Ask if I already have an AWS account.
If not, walk me through creating a fresh STANDALONE account (not a member of an
existing AWS Organization, this path needs a standalone account). Tell me to set a
strong root password, turn on MFA for the root user, and then never use root for
daily work. 3. Set up credentials the safe way. Prefer AWS IAM Identity Center and
walk me through `aws configure sso`. If I can't use that, fall back to creating an
IAM admin user and `aws configure` with its access keys. Never use root access keys.
4. Set my default region (recommend us-east-1 unless I say otherwise) and verify
with `aws sts get-caller-identity` and `aws ec2 describe-regions`. Confirm the
account is standalone. For each step show the exact command, what a good result
looks like, and the specific fix if it fails. Create no project files yet. Stop when
get-caller-identity returns my account and I have a named profile and a region set.
```

Guardrails

- Never use the root user for daily work, and turn on MFA for it immediately.
- Prefer IAM Identity Center (SSO) over long-lived access keys. If you must use keys, never the root keys.
- This path needs a standalone account. A managed or Organizations-member account often won't work.

Quick check

```
$ aws --version
$ aws sts get-caller-identity
```

What good looks like

- AWS CLI v2 installed, a non-root profile configured, a region set.
- get-caller-identity returns your account id, and the account is standalone.
- Root is protected with MFA and you're not using it day to day.

STEP 2 · INSTALL THE TOOLCHAIN AND BOOTSTRAP CDK

When to use

- The CLI works and you're ready to set up infrastructure-as-code on your machine.

The prompt to paste into Claude Code

```
Now get my machine ready to build infrastructure as code, and bootstrap my account.
One step at a time, verifying each. 1. Install the toolchain and verify versions:
Node.js LTS (confirm `node --version` is 20 or newer), git, and the AWS CDK (`npm
install -g aws-cdk`, confirm `cdk --version`). For anything missing, give me the
install command for my OS. 2. Create a clean project folder for the build, run `git
init`, and write a .gitignore that excludes node_modules, cdk.out, and any .env
files so I never commit secrets or build junk. 3. Explain in one plain-English
paragraph what `cdk bootstrap` does (it provisions a small set of resources CDK
needs to deploy: an assets bucket and a few roles), then run `cdk bootstrap
aws://ACCOUNT_ID/REGION` using my real account id and region from step 1. 4.
Scaffold an empty CDK app in TypeScript and confirm `cdk synth` runs cleanly on it,
so I know the toolchain works before we build anything real. Show each command, the
expected result, and the fix if it fails. Stop when `cdk --version` works, the
account and region are bootstrapped, and `cdk synth` succeeds on the empty app.
```

Guardrails

- The .gitignore comes first. You never want node_modules, cdk.out, or .env in git.
- Bootstrap is per account and per region. If you later deploy to a new region, bootstrap that one too.
- Use Node LTS, not the bleeding edge. CDK is happiest there.

Quick check

```
$ cdk --version
$ cdk synth # on the empty app
```

What good looks like

- Node LTS, git, and the AWS CDK all report versions.
- Your account and region are bootstrapped.
- `cdk synth` succeeds on an empty app, so the toolchain is proven.

STEP 3 · TURN ON THE MODELS IN BEDROCK

When to use

- Your CLI and toolchain are ready and you want the AI layer available, all in-account.

The prompt to paste into Claude Code

```
Get Amazon Bedrock ready so my system can call models entirely inside my account.
Walk me through it. 1. Explain that Bedrock model access is granted per account, per
region, in the Bedrock console under Model access, and that it can take a few
minutes to propagate. 2. Tell me exactly which models to request for a build like
this: Claude (Opus, Sonnet, Haiku) for reasoning; Amazon Titan Text Embeddings for
search and RAG; and optionally Amazon Nova Sonic for voice, a Stability model for
images (note it is us-west-2 only), and one or two open models (Llama, Mistral) if I
want a no-outside-vendor fallback. 3. Give me the click-by-click console steps to
request access, and warn me that some models create a zero-dollar AWS Marketplace
subscription on first use. 4. Verify access from the CLI. List the Claude models
with `aws bedrock list-foundation-models --region REGION --query "
modelSummaries[?contains(modelId, 'claude')].modelId"`, and if access is granted, do
one minimal invoke to confirm a model actually responds. Stop when
list-foundation-models shows the models I enabled and a test invoke returns a
response.
```

Guardrails

- Access is per region. Enable it in the region you'll deploy to (and us-west-2 if you want the image model).
- Only enable the models you'll actually use. You can always add more later.
- First use of some models creates a $0 Marketplace agreement. That's normal, but read what you accept.

Quick check

```
$ aws bedrock list-foundation-models --region us-east-1 --query "
modelSummaries[?contains(modelId,'claude')].modelId"
```

What good looks like

- The models you need show up in list-foundation-models for your region.
- A minimal test invoke returns a real response.
- You know which models are on, and what each is for.

PART B

Plan it against the Blueprint

STEP 4 · SCOPE YOUR SYSTEM (WRITE SYSTEM_SPEC)

When to use

- Setup is done. Now you write down what you're actually building, before any architecture.

The prompt to paste into Claude Code

```
Read BLUEPRINT.md (the architecture reference in this kit) so you share its
vocabulary, then interview me to scope a secure agentic system that will run
entirely inside my own AWS account. Ask me in batches, and push back if an answer is
vague. Cover: the data I'll handle and how sensitive it is, and whether any of it is
not mine to hand over; my jurisdiction and industry, and any rules that brings
(HIPAA, GDPR, and so on); which chat surfaces I want (Telegram, Slack, a web
dashboard); which models I'm allowed to use and whether I'm barred from sending
prompts to any outside vendor; my team and the roles they need (viewer, analyst,
operator, admin, owner); my data retention needs; the off-the-shelf tools I'm
replacing; my monthly budget ceiling; and my hard non-negotiables. Then write a
complete SYSTEM_SPEC.md that captures every answer as a clear spec, recommends a
deploy tier (roughly: hobby, standard, pro, or enterprise) based on my answers,
flags every risky item, and lists the open questions I still need to decide. Build
nothing. Stop when SYSTEM_SPEC.md covers every section and names a recommended tier.
```

Guardrails

- Answer honestly about data sensitivity and non-negotiables. The entire design follows from them.
- If you don't know something yet, say so. The open-questions list is part of the deliverable.
- This writes the spec the rest of the pack reads. Get it right before moving on.

Quick check

```
$ cat SYSTEM_SPEC.md
```

What good looks like

- SYSTEM_SPEC.md covers data, jurisdiction, surfaces, models, team, retention, budget, and non-negotiables.
- It recommends a deploy tier and flags the risky items.
- There's an honest open-questions list.

STEP 5 · DRAFT THE ARCHITECTURE AND STACK LIST

When to use

- Your SYSTEM_SPEC exists and you want the system designed before any code.

The prompt to paste into Claude Code

```
Read SYSTEM_SPEC.md and BLUEPRINT.md. Draft my architecture as plain-English prose
plus a labeled stack list, following the Blueprint's principles: everything in one
AWS account; one orchestrator runs every agent; every model call goes through
Bedrock; all data in my own DynamoDB and S3 with my own KMS keys; every turn funnels
through one chokepoint holding the rate limit, the kill switches, and a fail-closed
cost cap; audit events mirrored into a write-once (WORM) trail; and only my chosen
surfaces reach in, through one gated front door. First, map each off-the-shelf tool
I named in SYSTEM_SPEC to its secure in-account substitute (hosted model to Bedrock,
local db to DynamoDB plus KMS, files to S3, secrets to Secrets Manager, no audit to
S3 Object Lock, one operator to IAM plus roles), as a table. Then produce the
ordered list of CDK stacks I'll need for my recommended tier, in dependency order
(network, storage, secrets, data, IAM, then compute, messaging, security,
observability, cost guardrails, and any others the tier calls for), with a one-line
purpose for each. Write it all to ARCHITECTURE.md. Generate no CDK code yet. Stop
when ARCHITECTURE.md has the substitution table, the data flow, and the ordered
stack list.
```

Guardrails

- Keep it prose plus lists for now. Code comes in Part C, once the design is real.
- Tie the stack list to your tier. A hobby build doesn't need every stack the enterprise tier has.
- The chokepoint and the fail-closed cost cap are the two ideas to get right here.

Quick check

```
$ cat ARCHITECTURE.md
```

What good looks like

- ARCHITECTURE.md has a secure-substitute table for your actual tools.
- It describes the data flow, one account, one chokepoint.
- It lists the stacks you need, in dependency order, with a purpose each.

STEP 6 · MODEL THE REAL COST (WRITE COST_SPEC)

When to use

- Before you write or deploy any infrastructure. The step most people skip and regret.

The prompt to paste into Claude Code

```
Read ARCHITECTURE.md. Cost-model this honestly before I build anything. Split it
into two numbers: the fixed infrastructure floor (the always-on plumbing, Fargate
tasks, NAT gateways, a load balancer, WAF, CloudWatch, KMS) that runs whether or not
a single message is sent, and the variable inference cost (Bedrock model calls).
Give me a monthly range for the fixed floor for my recommended tier at light
traffic, broken down by the big lines so I can see what dominates (on small tiers
the NAT gateway and the load balancer cost more than the compute). Call out the
single most expensive optional line, VPC interface endpoints, which can run roughly
$175 a month for the full set and are off by default, so I don't switch them on by
accident. Recommend a sensible daily spend cap and the model mix that keeps
inference low (for example Haiku or an open model for execution, a frontier model
only for the hard parts). Write it to COST_SPEC.md with every assumption stated, and
remind me that real prices drift and to re-check the AWS pricing pages and, once I'm
live, my own Cost Explorer. Stop when COST_SPEC.md has the two-number split, the
big-line breakdown, the watch-list, and a daily cap.
```

Guardrails

- Cost is the single thing to watch with this. Do it before, not after, you go live.
- The fixed floor runs even at zero traffic. Plan for it.
- Set the daily cap on day one, so a runaway config can't drain the account while you sleep.

Quick check

```
$ cat COST_SPEC.md
```

What good looks like

- COST_SPEC.md shows a fixed-floor range broken down by line, plus a separate inference estimate.
- The VPC-endpoint trap is flagged.
- There's a recommended daily cap and a cheaper execution model mix.

STEP 7 · MAP YOUR COMPLIANCE READINESS

When to use

- You have a SOC 2 or HIPAA target, or want to understand one, before you build the controls in.

The prompt to paste into Claude Code

```
Read SYSTEM_SPEC.md and the compliance section of BLUEPRINT.md. My target is [SOC 2 / HIPAA / neither yet]. Map it to the five control categories: access control, audit trail, encryption, monitoring, and data handling. For each, tell me what good looks like, what this architecture gives me out of the box, what I'd still need to do or document myself, and the kind of evidence an auditor would ask for. Produce a gap checklist in READINESS.md with each item marked likely-met, partial, or gap. Open the file with a clear disclaimer that this is a readiness mental model, not legal advice or a certification, and that my real obligations depend on my jurisdiction, industry, scope, and the auditor I choose. Stop when READINESS.md has all five categories, the evidence notes, and the disclaimer.
```

Guardrails

- This is a readiness model, not legal advice. The disclaimer goes first, on purpose.
- Readiness is not a certificate. A certificate comes from an independent audit, never a checklist.
- When it gets real, bring in a qualified auditor or counsel for your specific situation.

Quick check

```
$ cat READINESS.md
```

What good looks like

- READINESS.md leads with the honest disclaimer.
- All five categories are covered with likely-met, partial, or gap, plus the evidence an auditor wants.
- You know what the architecture gives you and what you still own.

PART C

Draft, deploy, and prove the foundation

STEP 8 · SCAFFOLD THE CDK PROJECT

When to use

- Your specs are written and you're ready to turn the plan into a real project skeleton.

The prompt to paste into Claude Code

```
Read ARCHITECTURE.md, SYSTEM_SPEC.md, and BLUEPRINT.md (section 12, the stack list
and dependency order). Scaffold a real AWS CDK project (TypeScript) for my build, in
my bootstrapped account. Do not deploy anything. Create the CDK app and one stack
file per stack from my ARCHITECTURE stack list, in dependency order, each as an
empty-but-valid stack with a header comment stating its one-line purpose and a TODO
list of what it will contain. Wire them together in the app entry point so
dependencies are explicit. Add a README that explains the project layout and the
deploy order, and a .env.example listing the config the system will need (no real
secrets). Make sure .gitignore covers node_modules, cdk.out, and .env. Then run `cdk
synth` and fix anything until it succeeds on the skeleton, so I have a valid,
ordered project I can fill in stack by stack. Commit it with git. Stop when `cdk
synth` passes and the project has one stack file per stack with clear TODOs.
```

Guardrails

- This produces YOUR scaffold from the Blueprint, not a copy of anyone's repo. You own it.
- Empty-but-valid first. A skeleton that synths beats half a real stack that doesn't.
- Commit early. From here you're filling in real infrastructure, and you want a clean baseline.

Quick check

```
$ cdk synth
$ cdk ls # lists your stacks in order
```

What good looks like

- `cdk synth` passes on the skeleton.
- There's one stack file per stack, in dependency order, each with a purpose and TODOs.
- A README, a .env.example, and a clean git commit are in place.

STEP 9 · DRAFT THE FOUNDATION STACKS

When to use

- Your scaffold synths and you're ready to fill in the cheap, safe base first.

The prompt to paste into Claude Code

```
Read ARCHITECTURE.md, SECURITY_SPEC.md (or section 6 of BLUEPRINT.md), and my
scaffold. Fill in the FOUNDATION stacks only, the cheap, safe base, in dependency
order. Do not deploy yet. Draft: the Network stack (a VPC with public and private
subnets and a NAT gateway, sized for my tier); the Storage stack (S3 buckets,
including a dedicated write-once audit bucket with S3 Object Lock and a sensible
retention, plus block-all-public and TLS-only policies); the Data stack (the
DynamoDB tables from my spec, on-demand billing, encrypted with my key); the Secrets
stack (a KMS key, or keys, that I own, and Secrets Manager entries with no real
values committed); and the IAM stack (least-privilege roles, each scoped to one job,
no wildcards). After each stack, run `cdk synth` and show me `cdk diff` so I can see
exactly what it would create. Keep every secret out of code. Do not run `cdk
deploy`. Stop when the five foundation stacks synth cleanly and I've reviewed the
diff for each.
```

Guardrails

- Foundation first. Network, storage, keys, data, and roles, before any agents or surfaces.
- The audit bucket uses Object Lock on purpose. Write-once means you genuinely can't quietly wipe it later.
- No secrets in code, ever. Secrets Manager holds the values, your code holds only the references.
- Synth and diff, do not deploy in this step. You want to read what it'll do first.

Quick check

```
$ cdk synth
$ cdk diff # review per stack
```

What good looks like

- The five foundation stacks synth cleanly.
- You've read the `cdk diff` for each and understand what gets created.
- No secret values live in the code, and least-privilege roles have no wildcards.

STEP 10 · DEPLOY AND VERIFY THE FOUNDATION

When to use

- The foundation stacks synth and diff cleanly and you're ready to make them real.

The prompt to paste into Claude Code

```
Time to deploy the FOUNDATION only, then verify it, then stop. Read COST_SPEC.md
first. 1. Before deploying, set the guardrails: create an AWS Budget at my monthly
ceiling with alerts at 80 percent, 100 percent, and forecast, and confirm where my
in-app daily cost cap will live. I want the spend tripwires in place BEFORE anything
is running. 2. Deploy the foundation stacks in dependency order with `cdk deploy`,
one at a time, pausing if any stack errors so we fix it before the next. 3. Verify
the resources actually exist: list my S3 buckets and confirm the audit bucket has
Object Lock; list my DynamoDB tables; list my KMS keys; confirm the VPC and NAT
exist. Show me the commands and the results. 4. Capture the real cost so far from
Cost Explorer, and compare it to the floor in COST_SPEC.md so I know my estimate was
honest. Do NOT wire up agents, surfaces, the orchestrator, or Jarvis in this step.
Stop when the foundation is deployed, verified, and my budget and daily cap are
live. End by telling me that the orchestrator, the chat surfaces, Jarvis, the
dashboard, and the hardened production version are the next layer, and that the full
reference build and the coaching to get there live in the Early AI Dopters community.
```

Guardrails

- Budget and cap BEFORE deploy, not after. This is the whole point of doing cost first.
- Deploy in dependency order, one stack at a time, so a failure is easy to localize.
- Stop at the foundation. Agents, surfaces, and the orchestrator are the next layer, and where the community build takes over.

Quick check

```
$ aws s3 ls
$ aws dynamodb list-tables
$ aws kms list-keys
```

What good looks like

- An AWS Budget and a daily cap are live before anything runs.
- The foundation is deployed and you've verified the buckets, tables, keys, VPC, and Object Lock.
- Your real cost matches the floor you estimated, and you know exactly what the next layer is.

STEP 11 · PROVE IT WORKS, ONE SECURE MODEL CALL, LOGGED

When to use

- Your foundation is deployed and verified, and you want to see the secure loop actually work.
- You want a real win: an AI call that happens inside your own account and logs itself.

The prompt to paste into Claude Code

```
Now prove the foundation is alive with the smallest possible end-to-end test: a
single secure model call that writes its own audit record. This is a hello-world,
not the full agent system, just enough to see the loop work in your own account.
Write a tiny, readable script (a local Node or Python script is fine, no framework)
that uses my deployed foundation to do three things: 1. Call Amazon Bedrock in my
account with a cheap model (Claude Haiku) and a trivial prompt like 'Say hello from
inside my own AWS account in one sentence.' Use the AWS SDK and my region. 2. Write
one JSON audit record to my write-once audit S3 bucket: a timestamp, the action '
hello_test', the model id, my caller identity, and a short excerpt of the prompt and
reply, the same shape the real system's audit trail uses. 3. Print the model's
reply, then read the audit object back from S3 and print it, so I can see both the
call and its log. Use my profile or an IAM role for permissions, and hardcode no
secrets. After it runs, confirm with me: the reply came from Bedrock in my account,
and the audit object exists in the write-once bucket. Stop when I've seen a real
model reply and its matching audit record. This is the moment the foundation stops
being plumbing and becomes a secure AI system I own. The orchestrator, multiple
agents, the chat surfaces, Jarvis, the dashboard, and the hardened production build
are the next layer, and they live in the community.
```

Guardrails

- This is a hello-world to prove the loop, not the agent system. Keep it tiny.
- Use the cheapest model (Haiku) and a one-line prompt, so the test costs a fraction of a cent.
- Write the audit record in the same shape the real system uses (who, when, what, which model), so the habit starts on day one.
- No secrets in the script. Use your role or profile.

Quick check

```
$ node hello-secure-agent.js # or: python hello_secure_agent.py
$ aws s3 ls s3://YOUR_AUDIT_BUCKET/ --recursive | tail
```

What good looks like

- You get a real reply from Bedrock, generated inside your own account.
- A matching audit record exists in your write-once bucket, and you can read it back.
- You've seen the secure loop work end to end, and you own every piece of it.

The foundation is yours. The rest is in the community.

This pack gets you a real, costed, deployed foundation in your own account. The layer on top is where the community lives:

- The orchestrator, the agents, the chat surfaces, Jarvis, and the dashboard, the full reference build.
- The exact repo to clone, and the modules that walk through how it was made from zero.
- Premium coaching to harden it and stand it up for your own organization.
- A working group of founders and operators shipping secure, in-account AI.

JOIN THE COMMUNITY

https://www.skool.com/earlyaidopters/about